

BNCmatmul における LU 分解・前進後退代入の 分岐なし FMA+SIMD による高速化

— DS/TS/QS/DD/TD/QD 6 精度での実装と性能評価 —

2026 年 8 月 16 日 (改訂: 前進・後退代入の FMA+SIMD 化を追加)

概要

多倍長精度線形計算ライブラリ BNCmatmul の直接解法について, (1) LU 分解 (部分ピボット選択付き $PA = LU$) における後続部分行列の更新 $a_{jk} \leftarrow a_{jk} - a_{ji} a_{ik}$, および (2) 前進・後退代入 (三角方程式の求解) の内積計算を, 分岐なし融合積和演算 (branch-free FMA, arXiv:2607.11391) による融合演算に置き換え, その SIMD ベクトル版 (AVX2 / AVX-512 / NEON / SVE2) を DS, TS, QS, DD, TD, QD の全 6 実数精度に実装した。従来 SIMD 化されていなかった単精度系 (DS/TS/QS) では特に効果が大きく, NEON/SVE2 において LU 分解で DS 約 4.5–5.2 倍, TS 約 8–9 倍, QS 約 11–14 倍, 前進・後退代入でも DS 約 3.6–4.3 倍, TS 約 5–6 倍, QS 約 9–11 倍の高速化を達成した (次元 512–1024)。倍精度系 (DD/TD/QD) でも LU 分解 1.4–2.8 倍, 前進・後退代入は NEON/SVE2 で 2.1–4.0 倍 (serial で 1.1–1.8 倍) となった。良条件・ピボット付きの検証では数値精度は従来実装と同等かそれ以上であった。さらに Hilbert 行列・Frank 行列による悪条件事例でも, 誤差が条件数 $\kappa_{\infty}(A)$ に比例して増大する古典的挙動と, 精度破綻次元が従来版とほぼ一致することを確認した (誤差定数は問題依存で従来版と前後する)。

1 はじめに

BNCmatmul は, double 2/3/4 個で構成する DD/TD/QD 精度および float 2/3/4 個で構成する DS/TS/QS 精度をサポートする多成分型多倍長精度線形計算ライブラリである。先行して, 分岐なし DW/TW/QW-FMA アルゴリズム (arXiv:2607.11391, 以下「新 FMA」) がコンパイル時スイッチ `BNC_USE_NEW_FMA` の下でライブラリに統合され, AXPY・内積・GEMV・GEMM・多項式評価等の演算に配線済みであった。またその SIMD ベクトル版 (`_bncneon_dwfma`, `_bncavx512_qwfmaf` など, AVX2 / AVX-512 / NEON / SVE2 の 4 命令セット $\times 6$ 精度) が整備されていたが, LU 分解と前進・後退代入はこれらを利用していなかった。

本レポートでは, LU 分解および前進・後退代入へ新 FMA (スカラー版・ベクトル版) を適用した実装と, その正当性検証および性能評価の結果をまとめる。

2 手法

2.1 LU 分解: 後続部分行列更新の融合 FMA 化

部分ピボット選択付き LU 分解の計算量の主要部 ($\frac{2}{3}n^3$ 項) は, 各段 i における後続部分行列の更新

$$a_{jk} \leftarrow a_{jk} - a_{ji} a_{ik}, \quad j, k = i + 1, \dots, n \quad (1)$$

である。従来実装ではこれを多倍長乗算 $t \leftarrow a_{ji} a_{ik}$ と多倍長減算 $a_{jk} \leftarrow a_{jk} - t$ の 2 演算で計算しており, それぞれの演算内部で誤差項の正規化 (renormalization) を行うため冗長であった。

多成分型浮動小数点数の符号反転は各成分の符号反転のみで厳密に行えることから, 式 (1) は丸め誤差の増加なしに

$$a_{jk} \leftarrow \text{fma}(-a_{ji}, a_{ik}, a_{jk}) \quad (2)$$

と書き換えられる。新 FMA はこの 3 項演算を 1 回の融合演算として計算し, 演算量は DW/TW/QW でそれぞれ 17/66/146 flops, 誤差上界はそれぞれ $34u^2$, $184u^3$, $812u^4$ である。乗算 + 減算の 2 段構成に対して正規化パスが 1 回で済むため演算量が大きく削減され, さらに融合化により中間丸めが減るため精度はむしろ改善する。 $-a_{ji}$ は行 j の更新開始前に 1 回だけ符号反転して全 SIMD レーンへブロードキャストするため, 追加コストは無視できる。行方向に連続な a_{ik} , a_{jk} を SIMD レーンに載せ, 更新全体をベクトル FMA 1 回で計算する。

2.2 前進・後退代入: 行方向内積への再構成と融合 FMA 化

LU 分解後の求解は前進代入 $Ly = b$ と後退代入 $Ux = y$ からなり, 計算量はそれぞれ $O(n^2)$ である。後退代入

$$x_i \leftarrow \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right) / u_{ii} \quad (3)$$

は行 $u_{i,i+1:n}$ と解ベクトルの内積であり, 行優先格納ではメモリ連続アクセスとなるためそのまま SIMD 化できる。一方, 従来実装の前進代入は列方向アクセス (ℓ_{ji} , $j = i + 1, \dots, n$; スライド幅 = 行長) の AXPY 型更新であり, このままでは SIMD 化に不向きである。そこで新 FMA パスでは前進代入を行方向内積形式

$$y_j \leftarrow b_j - \sum_{i=1}^{j-1} \ell_{ji} y_i \quad (4)$$

に再構成し, 後退代入と同一の内積カーネルで計算する。

内積カーネルは全 6 精度共通の構成で, (1) SIMD レーンごとに独立な部分和をベクトル融合 FMA $\text{acc}_w \leftarrow \text{fma}(\ell_w, y_w, \text{acc}_w)$ で積算し, (2) 端数要素はスカラー新 FMA で積算, (3) 最後にレーン間の部分和をスカラー多倍長加算で水平還元し, (4) 全体を 1 回の多倍長減算で右辺から引く。除

算 (後退代入) はその後に 1 回行う。なお完全ピボット選択版 (SolveXXLSC) は行・列とも置換配列を介した不連続アクセスとなるため SIMD 化の対象外とし、スカラー新 FMA 化のみ行った。

レーン分割により総和順序が変わるため結果は従来実装と厳密には一致しないが、各部分和が融合 FMA で計算されるため誤差は同等かそれ以下である (第 4 節)。

3 実装

3.1 対象関数と変更ファイル

全 6 精度の以下のルーチンを対象とした ($xx \in \{DS, TS, QS, DD, TD, QD\}$)。

- `XXLUdecompPM`(部分ピボット・行実交換版): 後続部分行列更新を SIMD ベクトル FMA 化 (AVX2 / AVX-512 / NEON / SVE2 の 4 分岐 + スカラーフォールバック)。
- `XXLUdecomp` / `XXLUdecompP` / `XXLUdecompC` (ピボットなし/添字ピボット/完全ピボット版): スカラー新 FMA 化。
- `SolveXXLS` / `SolveXXLSP` / `SolveXXLSPM`(前進・後退代入): 行方向内積形式に再構成し、精度別共通内積カーネル `_bnc_xxsolve_dot`(SIMD 4 分岐 + スカラーフォールバック) で計算。
- `SolveXXLSC`(完全ピボット版): スカラー新 FMA 化。

変更・新規ファイルを表 1 に示す。DS 精度の LU 分解・求解はヘッダ宣言のみで未実装であったため、`src/dslu.c` を新規作成し、分解 4 種 (`DSLdecomp/P/C/PM`) と求解 4 種 (`SolveDSLS/P/C/PM`) を実装した。TS/QS 精度の `LUdecompPM` は従来スカラー実装のみであったため、SIMD 区間 (パディング済み `real_col_dim` 境界までの head 処理 + 整列 SIMD 本体) を新設した。

表 1 変更・新規ファイル一覧

ファイル	内容	種別
<code>src/ddlu.c</code> , <code>tdlu.c</code> , <code>qdlu.c</code>	LU 分解の FMA+SIMD 化 + 求解カーネル・Solve 系 FMA 化	変更
<code>src/tslinear.c</code> , <code>qslinear.c</code>	同上 (LU PM は SIMD 区間新設)	変更
<code>src/dslu.c</code>	DS 用 LU 分解・求解一式 (FMA+SIMD 付き)	新規
<code>src/Makefile.legacy</code> , <code>.am</code> , <code>.in</code>	<code>dslu</code> のビルド組込み	変更
<code>bench/lu/</code>	検証・ベンチマークハーネス一式	新規

3.2 コンパイル時スイッチによる互換性維持

すべての新パスはライブラリの既存方針に従い `BNC_USE_NEW_FMA` マクロでゲートした。同マクロを定義しないデフォルトビルドでは、編集後ソースから生成されるオブジェクトの `.text` セクションが編集前とビット単位で同一であることを `objcopy` により確認した (`scalar/NEON/SVE2`

構成)。すなわち既定動作は一切変化しない。

全 6 精度 $\times$ 5 命令セット構成 (scalar, NEON, SVE2, AVX2, AVX-512) $\times$ FMA 有無の全組み合わせについてコンパイル検証を行い (AVX2/AVX-512 は x86_64-linux-gnu-gcc によるクロスコンパイル検証), 全構成が成功した。生成済みライブラリアーカイブ (serial/NEON/SVE2 および AVX2/AVX-512) へは更新オブジェクトを反映済みである。

4 正当性検証

4.1 既知解との照合

対角優位な密行列 $A(a_{ii} = n, \text{非対角は } [0.1, 1.07) \text{ の擬似乱数, 全要素 float 厳密表現可能})$ と既知解 $x_i = (i \bmod 16) + 1$ から $b = Ax$ を作業精度で生成し, LUdecompPM+SolveLSPM による数値解の最大要素相対誤差を全 36 構成 (6 精度 $\times$ 3 バックエンド $\times$ FMA 有無) で測定した。DD/TD/QD/TS/QS では誤差ゼロ (厳密解を再現), DS では 10^{-13} オーダー (DS 単位丸め $u^2 \approx 3.6 \times 10^{-15}$ と次元数の積に相当) であり, 全構成が精度仕様どおりに動作した。

4.2 ピボット選択を要する行列での検証

新規実装の DS 系については, ピボット選択が実際に発生する $[-1, 1]$ 一様乱数行列 (次元 64) でも, 分解 + 求解 (いずれも新 FMA 版) を検証した (表 2)。ピボット付き 3 変種はすべて正常動作し, 新 FMA 版は従来比で同等以上の精度であった。NEON/SVE2 では求解内積のレーン分割により総和順序が変わるため serial と数値は一致しないが, 誤差は同桁 (9.6×10^{-12}) である。ピボットなし版の差は無選択 LU 固有の成長係数の揺らぎによるもので, どちらも正常な範囲である。

表 2 DS 精度 LU 分解 + 求解の最大相対誤差 (乱数行列, 次元 64, serial)

変種	従来 (mul+sub)	新 FMA(分解 + 代入)
DSLUDcompP(部分ピボット)	8.1×10^{-12}	2.0×10^{-12}
DSLUDcompC(完全ピボット)	7.1×10^{-12}	3.9×10^{-12}
DSLUDcompPM(行実交換)	8.1×10^{-12}	2.0×10^{-12}
DSLUDcomp(ピボットなし)	2.3×10^{-10}	2.1×10^{-10}

4.3 悪条件問題での検証 (Hilbert 行列・Frank 行列)

条件数が次元とともに指数的に増大する古典的な悪条件行列 2 種でも, LUdecompPM と SolveLSPM(新 FMA 版・従来版とも) を検証した。

- **Hilbert 行列** $a_{ij} = 1/(i+j+1)$ (0 始まり添字。 $\kappa_{\infty} \approx 2.3 \times 10^{59}$ at $n = 40$): 係数は QD

演算 (c_qd_div, FMA スイッチ非依存) で計算して各作業精度へ丸めて格納し, 同一精度の FMA 版・従来版が**厳密に同じ行列**を解くようにした。

- **Frank 行列** (上 Hessenberg 型, gallery('frank')) 相当。 $\kappa_\infty \approx 3.4 \times 10^{49}$ at $n = 40$): 成分は小さい整数で全精度において厳密に表現可能。

真の解を $x = (1, \dots, 1)^T$ とし, 右辺 $b = Ax$ は格納された係数から QD 演算で積算して作業精度へ丸めた。数値解の誤差も QD 演算で評価しているため, double 精度より遥かに小さい誤差 (10^{-60} オーダー) も測定できる。条件数 $\kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty$ は mpmath(120 桁) による参考値である。

図 1・図 2 に新 FMA 版 (serial) の最大相対誤差の次元依存性を, 表 3・表 4 に代表次元での従来版との比較を示す。全精度・全次元で誤差は理論どおり $\kappa_\infty(A) u_{\text{prec}}$ に比例して増大し (図の縦軸約 60 桁のダイナミックレンジにわたる指数的增长), 精度が破綻する次元は FMA 版・従来版でほぼ一致した (差は高々 2 次元)。NEON/SVE2 の誤差も serial と同桁である (総和順序の違いのみ)。

一方, 誤差の**定数係数**には差が見られる。新 FMA の誤差上界は DW/TW/QW で $34u^2/184u^3/812u^4$ と, 個別の乗算・減算 (それぞれ数 u^k) より大きく, 係数が厳密表現される Frank 行列ではこの定数差がそのまま観測される。特に QS では従来版比で最大 2-3 桁大きい誤差となる場合があった (表 4)。逆に良条件・ピボット付きの乱数行列 (前節) では FMA 版が上回るなど, 差は定数倍の範囲で問題依存であり, **誤差のオーダー ($\kappa_\infty u$) と精度破綻限界は変わらない**。誤差定数を優先する用途ではスイッチを無効化すれば従来演算に戻る。

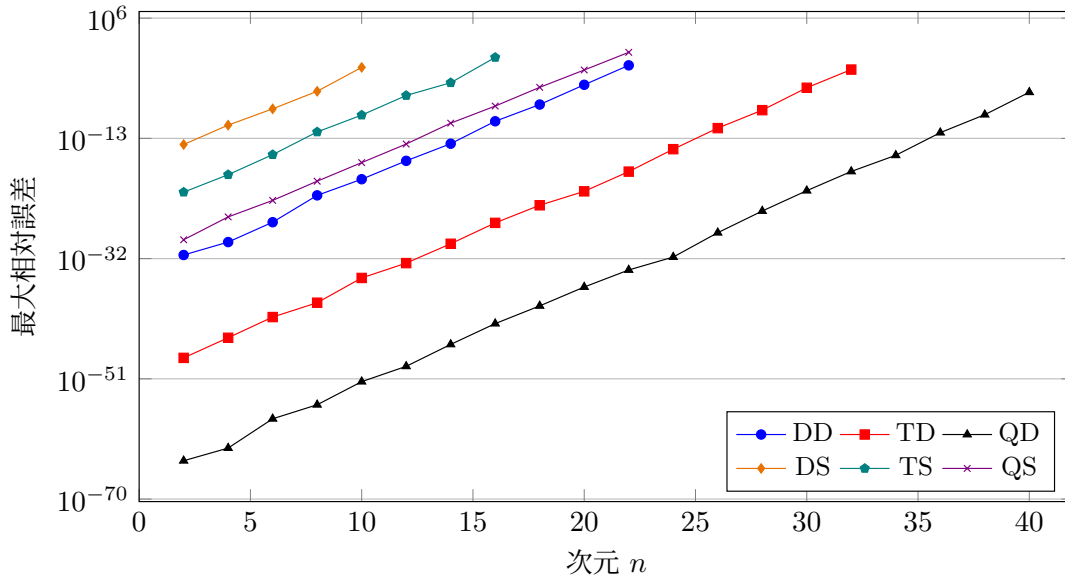


図 1 Hilbert 行列に対する最大相対誤差 (新 FMA 版, serial)。誤差 > 1 は省略

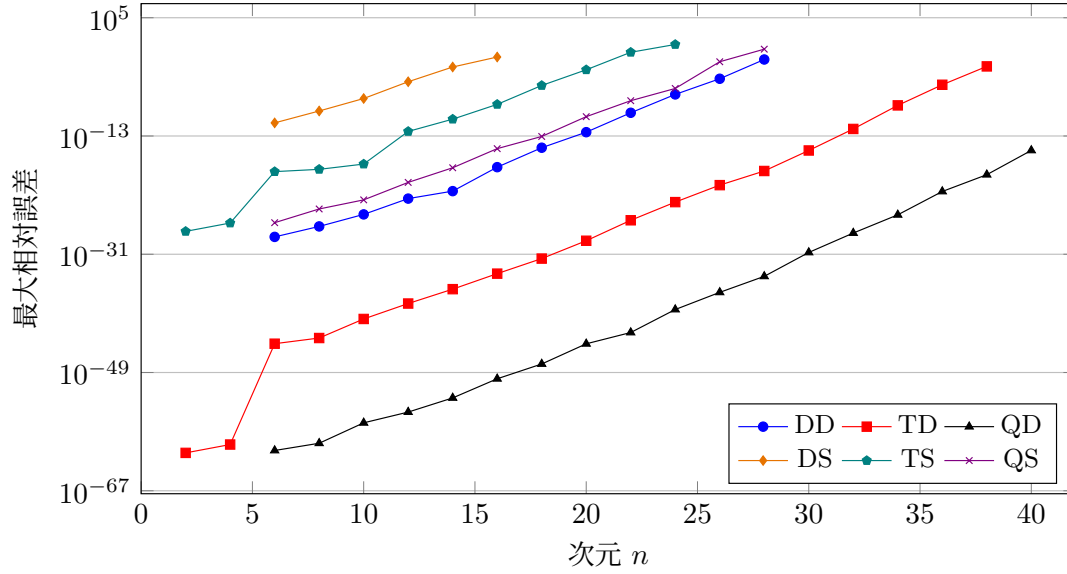


図2 Frank 行列に対する最大相対誤差 (新 FMA 版,serial)。誤差 > 1 は省略

表3 Hilbert 行列: 最大相対誤差の比較 (serial, 代表次元)

精度	n	$\kappa_{\infty}(A)$	従来 (mul+sub)	新 FMA
DD	12	4.1×10^{16}	7.0×10^{-18}	2.8×10^{-17}
DD	20	6.3×10^{28}	3.6×10^{-5}	2.9×10^{-5}
TD	20	6.3×10^{28}	1.2×10^{-21}	4.2×10^{-22}
TD	28	1.1×10^{41}	3.1×10^{-11}	2.9×10^{-9}
QD	28	1.1×10^{41}	1.3×10^{-25}	3.3×10^{-25}
QD	38	2.0×10^{56}	3.5×10^{-10}	5.6×10^{-10}
DS	6	2.9×10^7	1.1×10^{-8}	4.6×10^{-9}
DS	10	3.5×10^{13}	7.0×10^{-4}	1.7×10^{-2}
TS	8	3.4×10^{10}	3.2×10^{-13}	1.0×10^{-12}
TS	14	4.5×10^{19}	7.1×10^{-5}	6.2×10^{-5}
QS	12	4.1×10^{16}	5.6×10^{-16}	1.3×10^{-14}
QS	18	5.8×10^{25}	4.0×10^{-7}	1.2×10^{-5}

5 性能評価

5.1 評価環境と方法

- CPU: Arm Cortex-X925 + Cortex-A725(NVIDIA GB10, 計 20 コア), SVE ベクトル長 128 bit(NEON と同レーン数)
- コンパイラ: gcc 13.3.0, -O3 -ffp-contract=off(+NEON: -mcpu=cortex-a76, SVE2: -mcpu=neoverse-v2)

表 4 Frank 行列: 最大相対誤差の比較 (serial, 代表次元)

精度	n	$\kappa_{\infty}(A)$	従来 (mul+sub)	新 FMA
DD	12	6.9×10^9	9.0×10^{-24}	3.0×10^{-23}
DD	20	5.4×10^{19}	6.8×10^{-14}	3.8×10^{-13}
TD	20	5.4×10^{19}	4.7×10^{-30}	1.1×10^{-29}
TD	28	9.2×10^{30}	2.2×10^{-19}	4.7×10^{-19}
QD	28	9.2×10^{30}	1.1×10^{-35}	4.3×10^{-35}
QD	38	2.1×10^{46}	5.0×10^{-20}	1.3×10^{-19}
DS	6	6.4×10^3	2.4×10^{-13}	9.9×10^{-12}
DS	10	4.5×10^7	1.9×10^{-8}	5.1×10^{-8}
TS	8	4.3×10^5	4.0×10^{-19}	8.4×10^{-19}
TS	14	1.4×10^{12}	2.5×10^{-12}	3.6×10^{-11}
QS	12	6.9×10^9	7.6×10^{-22}	8.6×10^{-21}
QS	18	1.3×10^{17}	2.1×10^{-15}	8.5×10^{-14}

- 測定: `bench/lu/` ハーネス。LUdecompPM(行列複製コストは別測定して控除) と SolveLSPM(前進 + 後退代入) の実行時間をそれぞれ累積 0.2 秒以上になるまで反復して平均。1 コア逐次実行。

FMA 版バイナリは LU 分解・求解の翻訳単位のみを `-DBNC_USE_NEW_FMA` 付きで再コンパイルし、ライブラリより前にリンクした。それ以外のルーチン (行列初期化・行列ベクトル積等) は両版で共通のライブラリ実装を用いており、純粋に対象カーネルの差のみを比較している。

5.2 結果

次元 512 における高速化率 (従来比) を表 5 (LU 分解)・表 6(前進・後退代入) と図 3・図 4 に、全測定結果を表 7 に示す。

表 5 LU 分解の高速化率 (従来 mul+sub 比, 次元 512)

精度	serial	NEON	SVE2
DD	1.38	1.84	1.42
TD	1.90	1.61	1.71
QD	2.69	1.43	1.48
DS	1.40	4.56	5.20
TS	2.54	9.06	8.75
QS	3.80	13.53	12.26

表 6 前進・後退代入 (SolveLSPM) の高速化率 (従来比, 次元 512)

精度	serial	NEON	SVE2
DD	1.11	2.25	2.21
TD	1.36	2.61	2.64
QD	1.82	3.99	3.94
DS	1.08	3.97	3.99
TS	1.70	5.83	5.99
QS	2.83	10.89	11.24

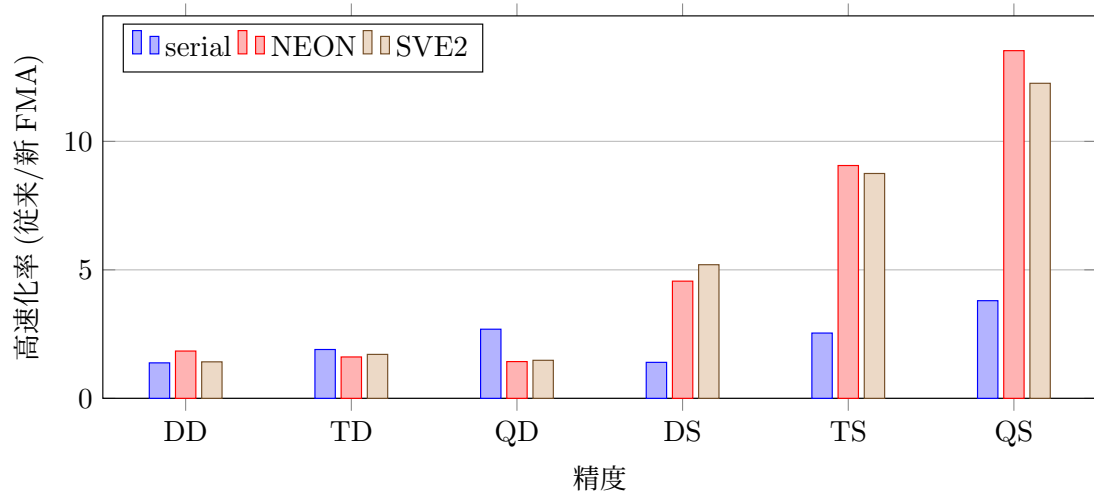


図 3 分岐なし FMA+SIMD による LU 分解高速化率 (次元 512)

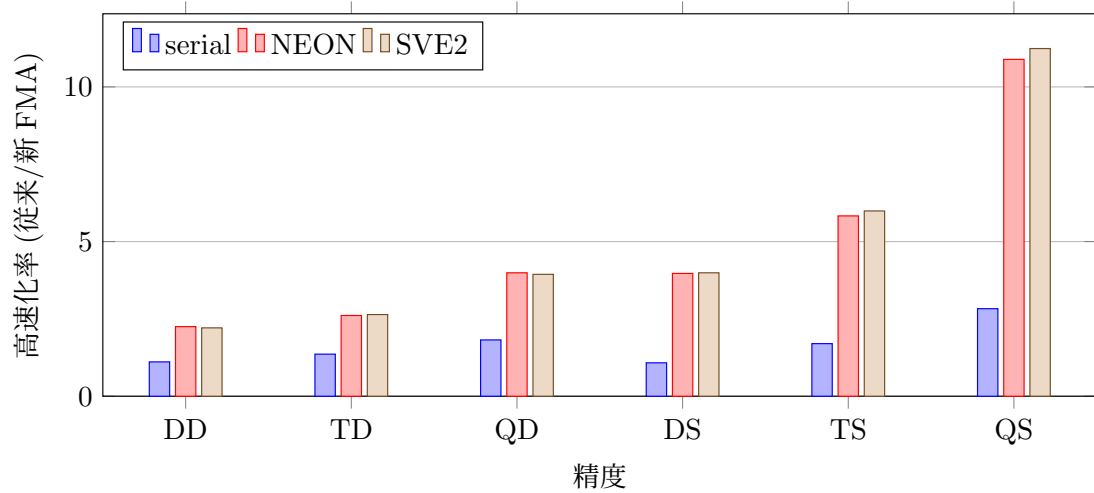


図 4 分岐なし FMA+SIMD による前進・後退代入高速化率 (次元 512)

表7 LU 分解・前進後退代入の全測定結果 (1 コア逐次)

精度	backend	次元	LU 分解 [s]			代入 [ms]			最大相対誤差
			従来	新 FMA	倍率	従来	新 FMA	倍率	
DD	serial	256	0.0095	0.0068	1.40	0.273	0.251	1.08	0
	NEON	256	0.0072	0.0039	1.85	0.270	0.124	2.17	0
	SVE2	256	0.0050	0.0035	1.42	0.265	0.128	2.07	0
	serial	512	0.0751	0.0543	1.38	1.118	1.004	1.11	0
	NEON	512	0.0569	0.0309	1.84	1.103	0.489	2.25	0
	SVE2	512	0.0402	0.0283	1.42	1.105	0.501	2.21	0
	serial	1024	0.5994	0.4339	1.38	4.515	4.024	1.12	0
	NEON	1024	0.4539	0.2503	1.81	4.440	1.943	2.28	0
	SVE2	1024	0.3207	0.2258	1.42	4.440	1.986	2.24	0
TD	serial	256	0.0548	0.0291	1.88	0.912	0.675	1.35	0
	NEON	256	0.0260	0.0164	1.58	0.907	0.364	2.49	0
	SVE2	256	0.0280	0.0168	1.67	0.921	0.371	2.48	0
	serial	512	0.4352	0.2293	1.90	3.632	2.675	1.36	0
	NEON	512	0.2040	0.1263	1.61	3.632	1.389	2.61	0
	SVE2	512	0.2225	0.1299	1.71	3.768	1.427	2.64	0
QD	serial	256	0.1990	0.0705	2.82	2.532	1.413	1.79	0
	NEON	256	0.0565	0.0401	1.41	2.504	0.666	3.76	0
	SVE2	256	0.0617	0.0424	1.46	2.498	0.675	3.70	0
	serial	512	1.4653	0.5456	2.69	10.120	5.546	1.82	0
	NEON	512	0.4285	0.2993	1.43	10.066	2.520	3.99	0
	SVE2	512	0.4718	0.3184	1.48	10.093	2.563	3.94	0
DS	serial	256	0.0092	0.0067	1.38	0.263	0.248	1.06	5.3×10^{-14}
	NEON	256	0.0092	0.0020	4.53	0.263	0.070	3.77	5.3×10^{-14}
	SVE2	256	0.0093	0.0019	4.96	0.266	0.073	3.63	5.3×10^{-14}
	serial	512	0.0748	0.0533	1.40	1.067	0.991	1.08	1.1×10^{-13}
	NEON	512	0.0735	0.0161	4.56	1.054	0.266	3.97	4.8×10^{-14}
	SVE2	512	0.0745	0.0143	5.20	1.066	0.267	3.99	4.8×10^{-14}
	serial	1024	0.5985	0.4328	1.38	4.477	4.022	1.11	3.2×10^{-13}
	NEON	1024	0.5907	0.1304	4.53	4.412	1.016	4.34	8.5×10^{-14}
	SVE2	1024	0.5969	0.1198	4.98	4.465	1.052	4.24	8.5×10^{-14}
TS	serial	256	0.0743	0.0300	2.48	1.149	0.686	1.67	0
	NEON	256	0.0734	0.0087	8.46	1.132	0.223	5.08	0
	SVE2	256	0.0735	0.0092	8.00	1.173	0.230	5.10	0
	serial	512	0.5932	0.2340	2.54	4.585	2.691	1.70	0
	NEON	512	0.5850	0.0646	9.06	4.517	0.775	5.83	0
	SVE2	512	0.6006	0.0686	8.75	4.814	0.804	5.99	0
QS	serial	256	0.2701	0.0696	3.88	3.878	1.402	2.77	0
	NEON	256	0.2880	0.0243	11.86	4.075	0.453	9.00	0
	SVE2	256	0.2908	0.0260	11.17	4.238	0.454	9.33	0
	serial	512	2.0413	0.5371	3.80	15.565	5.498	2.83	0
	NEON	512	2.2270	0.1647	13.53	16.393	1.505	10.89	0
	SVE2	512	2.2053	0.1799	12.26	17.013	1.514	11.24	0

5.3 考察

- **単精度系 (DS/TS/QS) の大幅な高速化:** 従来 LU 分解・求解ともスカラー実装のみだったため、SIMD 化 (float32 レーン数は倍精度の 2 倍) と融合 FMA 化の効果が乗算的に働き、LU 分解は QS で最大約 14 倍、前進・後退代入も NEON/SVE2 で DS 約 4 倍、TS 約 6 倍、QS 約 11 倍に達した。
- **倍精度系 (DD/TD/QD):** LU 分解は既に SIMD 化済みの実装に対する置き換えであるため、効果は融合 FMA による演算量削減分 (1.4–1.9 倍程度) である。成分数が多いほど乗算 + 減算の正規化コストが大きいため、スカラー (serial) では QD で 2.7–2.8 倍と最も効果が大きい。前進・後退代入は従来スカラーだったため、SIMD 化と合わせて NEON/SVE2 で 2.1–4.0 倍となった (serial では融合 FMA 分のみで 1.1–1.8 倍)。
- **代入の高速化率が LU 分解より小さい理由:** 代入は $O(n^2)$ でメモリ帯域律速に近く、また前進代入初期・後退代入末期の短い内積では水平還元・端数処理の固定費が相対的に大きいためである。次元が大きいくほど高速化率は改善する傾向にある。
- **NEON と SVE2 がほぼ同等**なのは、本評価環境の SVE ベクトル長が 128 bit で NEON とレーン数が同じためである。
- LU 分解の高速化率は次元 256–1024 でほぼ一定であり、更新カーネル律速であることと整合する。

6 まとめ

BNCmatmul の全 6 実数精度の LU 分解および前進・後退代入に分岐なし FMA+SIMD パスを実装した。BNC_USE_NEW_FMA 定義時に、LU 分解で最大約 14 倍 (QS/NEON)、倍精度系でも 1.4–2.8 倍、前進・後退代入で最大約 11 倍 (QS/SVE2) の高速化を、数値精度のオーダーを維持したまま達成した (良条件・ピボット付き乱数行列では従来比で改善、Hilbert・Frank 行列の悪条件事例でも誤差は $\kappa_{\infty} u$ に比例する古典的挙動を保ち、精度破綻次元は従来版とほぼ一致)。前進代入は列方向 AXPY 型から行方向内積型に再構成することで SIMD 化を可能にした。未実装だった DS 精度の LU 分解・求解一式も新規追加した。デフォルトビルドはビット単位で従来と同一である。

今後の課題として、(1) x86 実機 (AVX2/AVX-512) での実行性能評価 (本レポートではクロスコンパイルによる全構成コンパイル検証まで)、(2) ブロック LU (Strassen 型・OpenMP 並列版) 内部の LUdecomp_square 等への同様の適用、(3) 複素数精度 (CDD/CTD/CQD/CDS/CTS/CQS) への展開、が挙げられる。

参考文献

- [1] T. Kouya: “Performance evaluation of branch-free fused multiply-add algorithms for multi-component-type multiple-precision floating-point arithmetic”, arXiv:2607.11391v1, 2026.
- [2] BNCmatmul version 0.24, <https://na-inet.jp/na/bnc/>.